

JAVA INTERVIEW PREPARATION

CHAPTER 32

SPRING IOC AND DEPENDENCY INJECTION

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Spring IoC and Dependency Injection: Container Design, Boundaries, and Testable Objects

Senior / Lead Java Developer Reading Chapter

Spring's core value is not annotation scanning. It is inversion of control: application objects describe what they need, while a container creates those objects, supplies their collaborators, applies lifecycle policies, and can wrap them with infrastructure behavior. Dependency injection is the concrete mechanism most applications use to achieve that inversion.

From Object Construction to Object Collaboration

Without DI, a service constructs its own infrastructure:

```
final class OrderService {  
    private final OrderRepository repository = new JdbcOrderRepository();  
    private final PaymentClient payment = new HttpPaymentClient();  
}
```

The service is coupled to construction, configuration, credentials, timeouts, and implementations. With constructor injection, it owns only collaboration:

```
@Service  
final class OrderService {  
    private final OrderRepository repository;  
    private final PaymentClient payment;  
  
    OrderService(OrderRepository repository, PaymentClient payment) {  
        this.repository = repository;  
        this.payment = payment;  
    }  
}
```

```
configuration/container -> constructs repository  
                        -> constructs payment client  
                        -> injects both into OrderService
```

The container does not remove dependencies. It makes them explicit and relocates object-graph assembly to a composition root.

Bean Definitions Are Recipes

The `ApplicationContext` contains bean definitions describing type, construction, dependencies, scope, lifecycle callbacks, qualifiers, and infrastructure metadata. A definition is a recipe; the resulting object is the bean instance.

Beans can be discovered through component scanning or declared explicitly:

```
@Configuration
class PaymentConfiguration {
    @Bean
    PaymentClient paymentClient(PaymentProperties properties, HttpClient http) {
        return new HttpPaymentClient(http, properties.endpoint());
    }
}
```

Use component scanning for application-owned components with obvious construction. Use `@Bean` for third-party classes, conditional construction, named variants, or construction requiring deliberate code. A healthy system normally uses both.

Why Constructor Injection Is the Default

Constructor injection makes required dependencies visible, permits final fields, and prevents an object from existing in a partially wired state.

```
final class PricingService {
    private final PriceRepository repository;
    private final Clock clock;

    PricingService(PriceRepository repository, Clock clock) {
        this.repository = Objects.requireNonNull(repository);
        this.clock = Objects.requireNonNull(clock);
    }
}
```

The class can be tested without starting Spring:

```
PricingService service = new PricingService(fakeRepository, fixedClock);
```

Field injection hides dependencies, prevents ordinary construction of a valid object, encourages reflection-heavy tests, and makes excessive dependency count less visible. Setter injection remains appropriate for genuinely optional reconfiguration, but optional business dependencies often deserve an explicit strategy object instead.

Dependency Resolution

Spring usually resolves an injection point by type. If exactly one candidate exists, it is injected. Multiple candidates require a decision.

```
@Bean
@Primary
PaymentGateway primaryGateway() { return new StripeGateway(); }

@Bean("backupGateway")
PaymentGateway backupGateway() { return new BankGateway(); }
```

```
CheckoutService(
    PaymentGateway primaryGateway,
    @Qualifier("backupGateway") PaymentGateway backup
) { }
```

`@Primary` expresses a default among candidates. `@Qualifier` expresses semantic selection. Do not rely accidentally on parameter names or bean names when the domain has distinct meanings. Custom qualifier annotations make architectural intent clearer.

Collections and Optional Dependencies

Spring can inject all beans of a type:

```
FraudEngine(List<FraudRule> rules) {  
    this.rules = List.copyOf(rules);  
}
```

Ordering can be expressed with `@Order` or `Ordered`, but hidden global ordering can become fragile. For critical pipelines, an explicit configuration list is easier to review.

`Optional<T>`, `ObjectProvider<T>`, or nullable injection can represent optional availability. `ObjectProvider` supports lazy and repeated lookup:

```
ReportService(ObjectProvider<Exporter> exporter) {  
    this.exporter = exporter.getIfAvailable(NoOpExporter::new);  
}
```

Service-locator-style lookup throughout business code weakens dependency clarity. Keep dynamic lookup at integration boundaries.

Circular Dependencies Are a Design Signal

Constructor cycles cannot produce fully constructed immutable objects:

```
OrderService -> PaymentService -> OrderService
```

Modern Spring configurations commonly reject circular references, and constructor cycles fail clearly. `@Lazy` can defer one edge, but it usually hides a confused responsibility or missing domain service.

Break the cycle by extracting shared policy, publishing an event only when eventual processing is correct, moving orchestration to a higher-level service, or revisiting aggregate boundaries. Do not replace every cycle with application-context lookup.

Container Boundaries

`BeanFactory` provides fundamental creation and lookup. `ApplicationContext` adds enterprise facilities such as events, resource loading, message resolution, environment integration, and automatic recognition of many post-processors.

Most applications depend on `ApplicationContext` at startup but should not pass it into domain services. A service that calls `context.getBean()` hides dependencies and couples business logic to Spring.

```
framework boundary -> ApplicationContext and adapters  
application core   -> ordinary Java objects and explicit interfaces
```

This is not a ban on framework annotations. It is a rule that business decisions should remain understandable without simulating the container.

Configuration Classes and Proxy Semantics

`@Configuration` classes declare bean factories. In full configuration semantics, Spring can enhance the class so calls between `@Bean` methods return container-managed instances rather than arbitrary new objects.

Prefer method-parameter injection:

```
@Bean
OrderService orderService(OrderRepository repository) {
    return new OrderService(repository);
}
```

It is explicit and avoids depending on inter-method call enhancement. Configuration classes should remain assembly code, not repositories for business logic.

Profiles and Conditional Wiring

Profiles select groups of definitions for named environments, while conditional annotations and properties select definitions from actual capabilities.

```
@Bean
@Profile("development")
EmailSender consoleEmailSender() { return new ConsoleEmailSender(); }
```

Too many profile combinations create an untestable configuration matrix. Prefer a small number of environment profiles and typed properties for behavior. Production code should fail fast when a required dependency or credential is absent rather than silently activating an unsafe fallback.

Testing the Object Graph

Test business objects directly with constructors. Use focused Spring tests for wiring, conditions, proxies, serialization, security, and persistence behavior that actually depends on the framework.

unit test:	new Service(fake, clock)
slice test:	MVC, JPA, JSON, or security boundary
context test:	complete wiring and startup conditions
end-to-end test:	deployed behavior and real dependencies

Loading an entire context for every small rule slows feedback and can hide poor object design. Avoid mocking every collaborator through the container when ordinary construction would be clearer.

How the Container Resolves a Constructor

For a class with one constructor, Spring can use it without an `@Autowired` annotation. When several constructors are candidates, selection rules and required flags matter. A senior design avoids making construction ambiguous.

```
@Component
final class NotificationService {
    private final Sender sender;
    private final AuditSink audit;

    NotificationService(Sender sender, AuditSink audit) {
        this.sender = sender;
        this.audit = audit;
    }
}
```

The container conceptually resolves the graph from the outside inward:

```
NotificationService required
|
+-- resolve Sender candidate
|   +-- resolve Sender configuration
|
+-- resolve AuditSink candidate
    +-- resolve Audit storage/client
```

If a dependency is missing, Spring reports an unsatisfied dependency path. If several candidates match, it reports ambiguity. The useful part of the exception is often the deepest dependency path, because the top-level service may be several layers away from the missing definition.

Generic type information can participate in resolution. `Store<Order>` and `Store<Customer>` may be distinguishable injection candidates even though both implement the same raw interface. Do not erase that semantic information by injecting a raw `Store`.

Bean Names Are Identifiers, Not a Domain Model

Every bean has a name, but business logic should not become a switch over container names.

```
Map<String, PaymentGateway> gateways;
```

Injecting a map of named strategies can be useful at a composition boundary. The risk appears when HTTP input or database data directly chooses an arbitrary bean name. That exposes internal configuration as an external contract and can accidentally make new infrastructure reachable.

Prefer an explicit domain key-to-strategy registry:

```
final class GatewayRegistry {
    private final Map<PaymentRail, PaymentGateway> gateways;

    PaymentGateway forRail(PaymentRail rail) {
        return Optional.ofNullable(gateways.get(rail))
            .orElseThrow(() -> new UnsupportedPaymentRail(rail));
    }
}
```

The configuration layer builds the mapping; the business layer understands `PaymentRail` rather than Spring bean naming.

Container Extension Phases

Two extension families are often confused:

```
BeanFactoryPostProcessor -> changes bean definitions before ordinary beans are created
BeanPostProcessor        -> observes or replaces bean instances during creation
```

Property placeholder processing and configuration metadata work at the definition phase. Annotation injection and auto-proxy creation operate through instance-processing infrastructure. A custom definition post-processor that eagerly calls `getBean` can create application objects too early, before the full post-processor chain is registered.

Framework authors need this distinction. Application teams should normally prefer supported configuration APIs rather than custom container extensions, because extension ordering is global and startup-sensitive.

DI and Architectural Boundaries

Dependency inversion is strongest when the application core defines the capability it needs and an adapter implements it.

```
public interface PaymentAuthorizer {
    Authorization authorize(Payment payment);
}

@Component
final class HttpPaymentAuthorizer implements PaymentAuthorizer {
    // infrastructure implementation
}
```

```
controller adapter -> application service -> PaymentAuthorizer port
                                         ^
                                         |
                                HTTP provider adapter
```

Spring wires the direction; it does not create it. If the domain module imports a vendor SDK everywhere, adding constructor injection does not remove architectural coupling. Interfaces are valuable when they represent a real boundary, alternative, or test seam. Creating an interface for every class without such a reason adds indirection rather than inversion.

Optional Dependencies and Feature Absence

An optional bean and an optional business capability are different decisions. If audit publication is legally required, treating `AuditPublisher` as optional and silently substituting a no-op is unsafe. If metrics export is optional, a no-op implementation may be exactly right.

```
required capability missing -> fail startup
optional operational feature -> explicit no-op or conditional adapter
runtime feature unavailable -> defined degraded behavior or rejection
```

Make the fallback visible in startup logs and health information without exposing secrets. Silent fallback makes two identically deployed instances behave differently when one dependency is misconfigured.

Events Do Not Automatically Decouple Semantics

Application events can reduce direct knowledge between publishers and listeners, but synchronous listeners normally execute in the publishing thread. They can add latency, share the caller's transaction context, and fail the publishing call.

```
publisher -> ApplicationEventPublisher -> synchronous listener A
                                         -> synchronous listener B
```

Asynchronous listeners introduce executor, ordering, retry, and durability questions. An in-memory event is not a message broker and is lost on process failure. Use events when the coupling is genuinely event-shaped; do not use them merely to hide a circular dependency.

Required, Optional, and Deferred Resolution

An ordinary constructor dependency asks the container to resolve the collaborator while building the consumer. `ObjectProvider<T>` changes the time of resolution and can express optional, repeated, or lazy access. That is a lifecycle decision, not merely a way to suppress startup errors.

```
T constructor parameter    -> resolve now; exactly one suitable candidate
Optional<T>                -> zero or one candidate represented now
ObjectProvider<T>         -> resolve when provider is queried
List<T>                   -> all matching candidates, possibly empty
```

Deferred lookup is useful for a prototype factory, an expensive rarely used adapter, or integration code that legitimately tolerates absence. It is harmful when a required dependency is silently postponed until the first production request. Required capabilities should fail during startup with a clear configuration error.

Keep provider calls at the edge of the object graph. If every business method asks a provider or context for dependencies, the code has returned to service location and hides its true collaborators.

Injection Does Not Define Ownership Automatically

Receiving an object through DI does not mean the consumer owns its lifecycle. A shared HTTP client, executor, or [DataSource](#) is commonly container-owned and must not be closed by each service that injects it. Conversely, a factory may return a short-lived resource that the caller must close.

```
container creates singleton client -> container closes at context shutdown
service borrows database connection -> service closes/returns after operation
factory creates report session      -> caller owns explicit close
```

Document ownership in the abstraction and API. Lifecycle ambiguity produces double-close failures, leaked resources, and shutdown races even when dependency resolution is correct.

Candidate Selection Should Encode Meaning

When several beans share a type, choosing one by [@Primary](#) is appropriate only when a genuine default exists. If both fraud engines must run, inject a collection. If the selection depends on payment rail, model a registry. If two clients target different regions, use semantic qualifiers.

```
@Qualifier
@Target({FIELD, PARAMETER, METHOD, TYPE})
@Retention(RUNTIME)
@interface SettlementRegion {
    String value();
}
```

Custom qualifiers reduce accidental dependence on method or parameter names. They also make tests and configuration reviews speak the domain language. Bean-name fallback can be convenient, but renaming a factory method should not silently alter a critical routing decision.

Component Scanning Is an Architectural Input

Scanning is a bean-definition discovery mechanism, not a module boundary. If a root package accidentally includes test utilities, migration tools, or a second adapter, the context may create unexpected beans or ambiguities.

```
application root
+-- orders.application    intended services
+-- orders.adapters       intended adapters
+-- experiments           accidental scan candidate
```

For a modular codebase, keep scan roots explicit and use configuration imports to assemble modules deliberately. A module should expose a small configuration surface rather than requiring consumers to know every implementation package. Context-startup tests can verify that forbidden dependencies and duplicate adapters are absent.

Proxies Change the Exposed Runtime Object

Post-processors may expose a proxy implementing the bean's interfaces or subclassing its concrete type. Injection usually resolves the exposed object, not necessarily the raw instance constructed by the factory.

```
bean definition -> raw service -> post-processing -> transactional/security proxy
                                                    -> injected into caller
```

This matters for concrete-type lookup, equality assumptions, final methods, and self-invocation. The IoC lesson is to inject and program to the capability contract. Do not reach behind the proxy to call the target directly; doing so bypasses infrastructure policy. Proxy mechanics do not excuse large interfaces, but stable interfaces make boundaries and testing clearer.

Startup Failure as a Dependency Graph Report

Spring creation errors are often nested because the failing bean was needed several levels below the entry point.

```
controller creation failed
-> order service creation failed
    -> payment client creation failed
        -> PaymentProperties validation failed: endpoint missing
```

Read from the deepest meaningful cause upward. Record bean name, injection point, qualifiers, property origin, and whether the bean was created early by infrastructure. Fixing the top-level `UnsatisfiedDependencyException` message without following the chain often produces random annotations and duplicate beans.

For slow startup, measure definition count, scanning, post-processor phases, eager initialization, migrations, and external calls. Spring's startup instrumentation can expose phases; ordinary business logging should not guess which bean consumed the time.

A Design Review Checklist for the Object Graph

Before approving wiring, ask:

```
Are required dependencies visible in constructors?
Does each ambiguity have a semantic selection rule?
Are optional capabilities genuinely safe to omit?
Does dependency direction follow module boundaries?
Who owns shutdown of every long-lived resource?
Does any business object look up the container?
Can core behavior be tested without starting Spring?
```

These questions shift review from annotation syntax to object-graph correctness. A container can assemble a poor architecture perfectly; the graph still needs human design.

A Strong Interview Explanation

When asked why constructor injection is preferred, a senior answer should go beyond "it is easy to test." Constructor injection defines a valid-state boundary: the object cannot exist without required collaborators, dependencies can be final, cycles are exposed during graph construction, and the class remains usable without Spring. Field injection performs mutation after construction and hides the public construction contract.

When asked whether Spring creates tight coupling, distinguish coupling to a framework from coupling between application concepts. An `@Service` annotation is usually light framework metadata. Calling `ApplicationContext` throughout the domain is behavioral coupling. More importantly, DI makes existing

dependencies visible; good interface and module design determine whether those dependencies point in the correct architectural direction.

Production Perspective

Startup failures often reveal missing candidates, duplicate candidates, circular graphs, invalid configuration, or work performed too early during construction. Read the deepest cause and the dependency path, not only the top `BeanCreationException`.

Large contexts increase startup time and make accidental scanning more likely. Place the application entry point near the intended package root, define explicit scan boundaries for modular systems, and use condition reports when auto-configuration participates.

Interview-Ready Summary

IoC moves control of object creation and wiring to the container. DI supplies explicit collaborators, with constructor injection preferred for required dependencies and testability. Bean definitions are recipes; the `ApplicationContext` creates scoped instances and applies container services.

Resolve ambiguity intentionally with qualifiers and primary candidates. Treat circular dependencies as design feedback. Keep `ApplicationContext` lookup out of business services, use explicit configuration at the composition root, and test ordinary objects without Spring where container behavior is irrelevant.

Revision Notes

- IoC moves object assembly and lifecycle control to a container.
- DI makes collaborators explicit; it does not eliminate dependencies.
- Bean definitions are recipes for managed instances.
- Constructor injection is preferred for required dependencies.
- Final fields prevent partially configured required state.
- Field injection hides dependencies and complicates ordinary tests.
- Use `@Bean` for deliberate or third-party construction.
- Type resolution needs qualification when several candidates exist.
- `@Primary` supplies a default; `@Qualifier` expresses selection.
- Collections can inject all strategy implementations.
- `ObjectProvider` supports optional or lazy lookup at boundaries.
- Constructor circular dependencies are usually design problems.
- `@Lazy` can defer a cycle but does not improve responsibility boundaries.
- `ApplicationContext` adds facilities beyond `BeanFactory`.
- Avoid using the container as a service locator inside business logic.
- Keep configuration classes focused on assembly.
- Limit profile combinations and fail fast on unsafe configuration.
- Unit-test ordinary objects directly; test Spring when Spring behavior matters.
- Diagnose startup errors through the complete bean dependency path.
- Provider-based lookup changes resolution timing and should remain at boundaries.
- Required capabilities should fail during startup rather than at first use.
- Injection does not automatically transfer resource ownership.
- Use semantic qualifiers when multiple implementations have distinct roles.
- Component scanning discovers definitions; it does not enforce modules.
- Callers normally receive the post-processed proxy, not the raw target.
- Review dependency direction, ambiguity, optionality, and shutdown ownership together.